TASK 1: REQUIREMENT ANALYSIS & CUSTOMER DATABASE MODULE

PROJECT DETAILS

- **Name:** Kaviya S
- **Roll No:** E25AI118
- **Department:** B.Sc Computer Science With AI

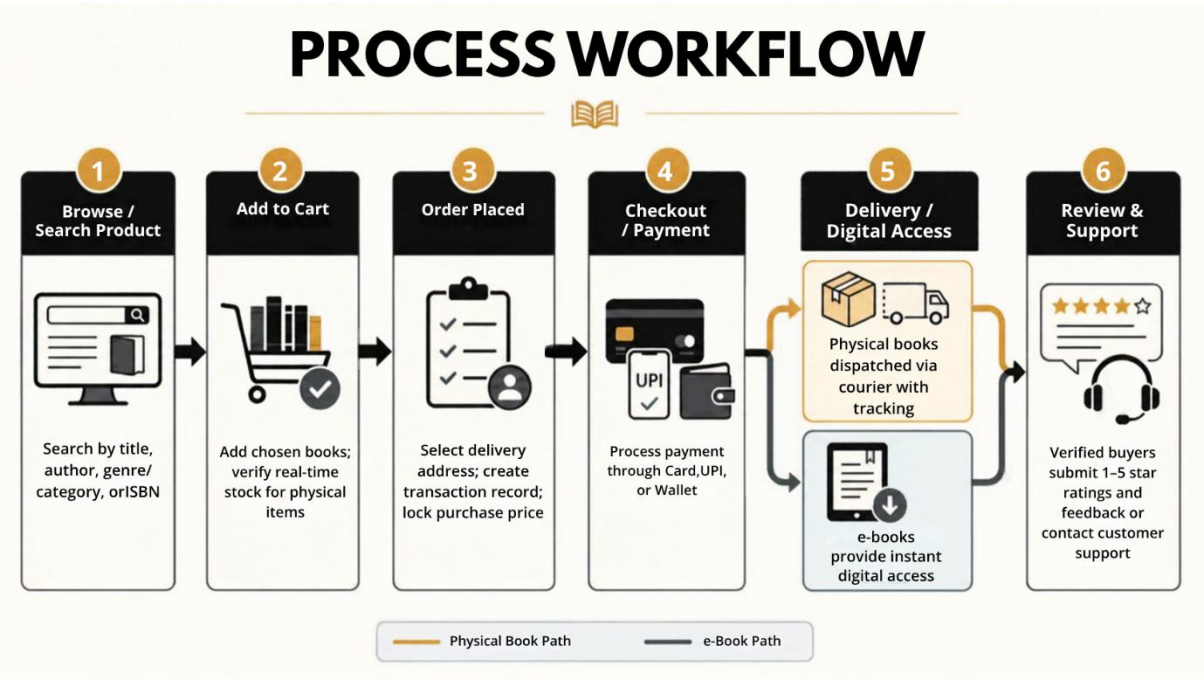
1.REQUIREMENT ANALYSIS

1.1 Introduction & System Overview

As our e-commerce bookstore continues to experience rapid growth, a centralized Database Management System (DBMS) is required to handle high volumes of transactional data, track catalog inventories, process orders, and securely manage user profiles. The system serves as the primary backend for a hybrid bookstore platform that supports both physical books (paperbacks and hardcovers) and digital assets (e-books). The database must ensure strict data integrity, eliminate redundancy, and support real-time operations alongside business intelligence analytics.

1.2 Bookstore Business Process Flow

The transaction lifecycle of the e-commerce bookstore consists of six core operational stages:



- **Browse / Search Product:** Customers search the catalog for books by title, author, genre/category, or ISBN.
- **Add to Cart:** Customers add chosen books to their shopping cart while the system verifies real-time stock levels for physical items.
- **Order Placed:** The customer selects a delivery address and places the order, creating a transaction record that locks in the purchase price.
- **Checkout / Payment:** Handles financial transactions through payment gateways (Card, UPI, Wallet).
- **Delivery / Digital Access:** Physical goods are dispatched via couriers with tracking numbers, while e-books grant instant digital access, bypassing logistics.
- **Review & Support:** Verified buyers submit star ratings (1–5 scale) and feedback for purchased books or seek customer support.

1.3 System Entity-Relationship Matrix

Entity Name	Description	Key Attributes	Key Relationships
CUSTOMER	End-user purchasing books on the platform	user_id (PK), first_name, last_name, email, hashed_password, phone_number, account_creation_date	1-to-Many with ADDRESS, ORDER_MASTER, and REVIEW
ADDRESS	Saved billing and delivery locations.	address_id (PK), user_id (FK), street, city, state, pincode, address_type	Many-to-One with CUSTOMER
BOOK	Master catalog item for physical books and e-books.	book_id (PK), isbn, title, format, unit_price, page_count	1-to-Many with ORDER_ITEM, INVENTORY, and REVIEW
ORDER_MASTER	Overall checkout transaction record.	order_id (PK), user_id (FK), order_datetime, total_amount, status	Many-to-One with CUSTOMER, 1-to-1 with PAYMENT
PAYMENT	Monetary transaction record tied to an order.	payment_id (PK), order_id (FK), payment_method, payment_status, final_billed_amount	1-to-1 with ORDER_MASTER, 1-to-1 with INVOICE
REVIEW	Customer ratings and feedback for purchased books.	review_id (PK), book_id (FK), user_id (FK), rating_value, review_text, timestamp	Many-to-One with CUSTOMER and BOOK

2. DATABASE TABLE CREATION (CUSTOMER MODULE)

2.1 Customer Table Creation:

```
SQL> CREATE TABLE customer (
  2      customer_id VARCHAR2(20) PRIMARY KEY,
  3      first_name  VARCHAR2(20),
  4      last_name   VARCHAR2(20),
  5      email       VARCHAR2(30),
  6      phone       NUMBER(10),
  7      created_at  DATE
  8  );

Table created.

SQL> DESC CUSTOMER;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 CUSTOMER_ID                               NOT NULL VARCHAR2(20)
 FIRST_NAME                                         VARCHAR2(20)
 LAST_NAME                                          VARCHAR2(20)
 EMAIL                                              VARCHAR2(30)
 PHONE                                              NUMBER(10)
 CREATED_AT                                         DATE
```

2.2 Insert Values:

i. Direct Insert (First Customer)

```
SQL> INSERT INTO customer
  2  VALUES ('CUST-101', 'Ananya', 'Roy', 'ananya.reads@gmail.com', 9876543210, TO_DATE('2026-01-15', 'YYYY-MM-DD'));

1 row created.
```

ii. Interactive Insert via Prompt Variables

```
SQL> INSERT INTO customer (customer_id, first_name, last_name, email, phone, created_at)
  2  VALUES (
  3    '&cust_id',
  4    '&first_name',
  5    '&last_name',
  6    '&email',
  7    &phone_num,
  8    DATE '&created_date'
  9  );
Enter value for cust_id: CUST-102
old   3:   '&cust_id',
new   3:   'CUST-102',
Enter value for first_name: Rohan
old   4:   '&first_name',
new   4:   'Rohan',
Enter value for last_name: Mehta
old   5:   '&last_name',
new   5:   'Mehta',
Enter value for email: rohanmehta@gmail.com
old   6:   '&email',
new   6:   'rohanmehta@gmail.com',
Enter value for phone_num: 8765432109
old   7:   &phone_num,
new   7:   8765432109,
Enter value for created_date: 2026-02-10
old   8:   DATE '&created_date'
new   8:   DATE '2026-02-10'

1 row created.
```

Like this we can create as many rows as we want.

iii. Displaying Records

```
SQL> SELECT * FROM customer;

CUSTOMER_ID          FIRST_NAME           LAST_NAME
-------------------- -------------------- --------------------
EMAIL                             PHONE CREATED_A
------------------------------ ---------- ---------
CUST-101             Ananya               Roy
ananya.reads@gmail.com         9876543210 15-JAN-26

CUST-102             Rohan                Mehta
rohanmehta@gmail.com           8765432109 10-FEB-26

CUST-103             Priya                Nair
priya.nair@gmail.com           9087654321 28-FEB-26


CUSTOMER_ID          FIRST_NAME           LAST_NAME
-------------------- -------------------- --------------------
EMAIL                             PHONE CREATED_A
------------------------------ ---------- ---------
CUST-104             Vikram               Seth
vikram.seth@gmail.com          8901234567 12-MAR-26
```

iv. Update

```
SQL> UPDATE customer
  2  SET phone = 9812345678
  3  WHERE customer_id = 'CUST-104';

1 row updated.
```